

CIS*3700 - Assignment 5 Report

By Christopher Lozinski

ID: 1237945

2 April, 2026

My work with AI on this Assignment (Q1/Q2)

For this assignment, I chose Claude Sonnet 4.6 as my primary AI partner because of its proficiency in handling long, technical specifications and its ability to generate structured code across multiple files. Additionally I wanted to use it outside of my IDE so I used the web version so it didn't have access to directly editing my files and I could read over all its changes and compare it to what I have before implementing specific line by line changes. The process began with me providing the AI with the complete assignment PDF to ensure it understood the overarching logic of, for example, the log analysis tool and explaining the general implementation process I had envisioned for this assignment (i.e how many files I knew I wanted to build, how I wanted each to work generally and the specific rules that needed to be implemented, providing specific and targeted information for all).

After it understood, we walked through each program building out the functionality function by function, I would read over its work and then bring it into my IDE and continue, moving function by function through each file. Our workflow followed the structured pair-programming approach that was outlined in the readings within the assignment, frequently swapping roles to maximize efficiency and correctness as well. With Claude initially in the Driver role, focusing on the immediate task of writing the syntax, I served as the Navigator in order to observe the code for potential logical errors and ensure that the way it was implementing the functions aligned with guidelines and requirements of the assignment.

As the process went on, I had to often take on the driver role to implement specific changes that the LLM struggled to do properly. One specific example of this was when the AI failed to follow the structure of the outputs for the tests, hallucinating that there were no spaces at all in the output. Knowing this was not correct, I refused its changes, corrected the behaviour tediously and manually edited the formatting to adjust the output so it matched the requirements. This role-swapping was essential because while the AI was fast, it frequently introduced really "stupid" bugs that required someone who fully understood the guidelines to correct them, being hyper vigilant.

How well was I able to complete this assignment in this format? (Q2 cont)

I was able to complete the core logic of the assignment relatively well, having the correct identification of all the complex violations. However the work flow was... far from perfect. While the final tool successfully handles the core logic of the log analysis, I had to frequently steer the ship, so to speak, in the right direction to ensure the code was not deviating in a negative feedback loop from the assignment.

As I mentioned, while the AI functioned well as a Driver for generating the skeleton of the main file, I had to act as a rigorous Navigator to point out that its output was deviating from the literal specification. To build on the example I mentioned prior, the AI repeatedly attempted to indent sub-bullets in the NOTES section with a tab and extra print lines, even after I corrected it, it would tell me that I was the one that was incorrect. So, although the final software I achieved was functionally accurate as per the requirements, to get to this point required significant manual intervention and iterative correction.

Comparing solo development to pair programming (Q3)

Comparing this experience to if I had done this through solo development, I would say that the results were mixed and I would describe my feelings on them as neutral. If I had completed the assignment on my own, it likely would have taken much longer to draft the initial function communication and I probably would have made a mistake with some sort of syntax writing or with the high level planning. These are errors I encounter when I develop on my own, for example realizing I should've implemented things in a different way so I would have to go back and refactor everything to fit what I realized after the fact was the better way or more correct way to achieve my goals.

On the other hand though, had I done it solo, it would have been considerably less frustrating because I would have avoided the really stupid and nonsensical bugs / formatting errors that the AI introduced for seemingly no reason at all. While the AI helped me avoid missing certain high-level architectural points, it also forced me to spend a disproportionate amount of time fixing minor errors that I would not have made myself. Our effectiveness as a pair was

hindered by the AI's context and visual blindness, something that I know many companies are actively working to try and resolve within their models.

I wouldn't say it made my work worse, since the pair-programming definitely improved the speed of my initial implementation, it definitely made the final stretch / refinement process a lot worse by introducing a lot of unnecessary friction.

What I learnt from this experience (Q4)

The most significant lesson I learned from this pair programming experience is that AI can be an excellent tool for high level functionality and planning, helping me to realize my ideas and architect things a lot quicker. It was often unable to grasp the nuance of specific, detailed directions though, as I discovered that there is a communication gap where the AI assumed that I was misinformed even when as the developer, I was providing it a direct correction based on the assignment spec, it would second guess me, which in an IDE environment I think can be really dangerous for a code stacks maintainability. As well, I would say this experience taught me that while pair programming does have the benefits I described above, it should always, no matter what, have a human partner nearby cross checking all the work it suggests, remaining hyper vigilant for what needs to be achieved and avoid significant deviations from a standard.

The AI & My Strengths and Weaknesses (Q5 & 6)

Claude's greatest strengths were its speed, making syntax errors like a simple semi colon breaking a program become pretty much impossible to achieve in this work flow, as well as its ability to plan out the implementation of complex, multi step functions more quickly than a regular dev could. In the beginning it was particularly useful for breaking down how to parse the logs and identify specific violation types and bring my abstract implementation ideas to life. Its weaknesses were its overconfidence and its tendency to hallucinate, adding things I didn't ask or request it to, then just scrap everything and give up when I would try to correct it. It could get caught in a thought loop where it would question my request and what it should be doing leading to long wait times for a simple response that I know would have been faster, at that point, for me to just do manually.

I would say that my greatest strength during this process was my attention to detail and my ability to keep the end goal of the development in mind. I knew what the direction of this assignment had to be and because of that I was able to identify exactly when it would stray from what I needed it to do and correct it. As I have mentioned, my weaknesses were perhaps not delegating the responsibilities more clearly. If I had understood the limitations of this LLM beforehand, I would have not waited for its responses or even asked it to do certain tasks and just did them myself entirely. Because I didn't do this, I ended up spending more time correcting really simple mistakes and suggestions, eventually just having to do it myself, meaning the whole time and back and forth process was a waste of time. I know that I would have switched into the Drivers seat a lot sooner in some sections and will definitely do so in the future the more these tools seep into the work flow of all programmers.

Advice to developers of my AI to improve it's Pair Programming (Q7)

If I was to give advice to Anthropic to develop Claude, the most important advice that I would say I have is to improve its ability to strictly adhere to provided text and examples. To listen to the driver at all costs and if it has doubts instead of getting stuck in a thought loop, to just simply prompt the user for clarification as to not waste time. As well, stating the limitations of the software would have been immensely helpful, so that I didn't have to learn that through countless agonizing minutes of waiting. Claude and LLMs in general definitely need better mechanisms for recognizing when a user is correct based on a literal specification that they have provided, as it currently has a tendency to seemingly prioritize its pre-trained patterns over, for example, a user's specific, detailed directions. Perhaps even adding in a feature for the bot to switch into a mode such as Driver or Navigator to accomplish different tasks, as I have seen some interfaces somewhat integrate, would be really helpful. I found a lot of the time it would also output huge swaths of code and not highlight the changes it made specifically causing me to go line by line and determine what the differences were since even when I would ask it what they were it would be entirely clear where it was referencing. While I know this is a feature in some IDE LLMs, I find using an IDE LLM ,especially after this experience, to be quite risky to a project as it might make or revert changes and cause greater frustration to a programmer. I think separation between the working code base and the LLM is beneficial for both parties and perhaps a mirrored version of the repository for the AI to work in that would compare to the active user's version, like a branch, could help in resolving these problems.